EMBEDDED LINUX SYSTEMS / LAB REPORT

LAB-03 Debugging & Performance Analysis

Remote debugging • Memory safety • CPU profiling • Real-time latency

Overview. This lab develops and evaluates a reproducible Embedded Linux workflow across an AlmaLinux x86_64 host and a QEMU ARMv7 target. Four experiments investigate program correctness, memory management, CPU behavior and scheduling latency.

Experimental environments

HOST / ANALYSIS	ARM / TARGET
AlmaLinux 9.8, x86_64 **GCC, GDB, Valgrind, strace, perf, cyclictest and stress-ng**	QEMU vexpress-a9 / Cortex-A9 ARMv7-A, Linux 5.15, 512 MB RAM and BusyBox initramfs

Results at a glance

3	16,900 B	51.39%	3,919 µs
C errors localized with GDB	Memory leaks fixed in Valgrind test	heavy_compute() CPU sample overhead	Peak latency under stress

Evidence: C sources and ARM/x86 binaries; raw GDB, Valgrind, strace, perf and cyclictest logs; a latency histogram; and report/Lab03_Report.pdf.

Measurement boundary: x86 host results, ARM QEMU behavior and real-time host latency are separate experiments and are not treated as interchangeable.

01 / ENVIRONMENT

System Architecture & Project Files

One source tree supports host-native testing and cross-compiled ARM execution. The host manages compilation, debugging and analysis; the QEMU guest provides an ARM Linux target for binaries and syscall inspection.

Host-to-target workflow

```
C source files
   |-- gcc (x86_64) --------> Valgrind / strace / perf
   |
   `-- ARM cross-compiler --> ARM executables
                                  |
                            BusyBox initramfs
                                  |
                           QEMU ARMv7 / Linux 5.15

Host GDB <--- UART forwarded to TCP :1234 ---> ARM gdbserver
Host cyclictest + stress-ng ---> scheduling latency measurements
```

Target specification

Component	Configuration
Emulated board / CPU	QEMU vexpress-a9 / Cortex-A9
Architecture / kernel	ARMv7-A / Linux 5.15
Guest RAM / rootfs	512 MB / BusyBox initramfs
Console / debug	PL011 UART / ARM gdbserver
Guest observation	/proc/<PID>/status and /proc/<PID>/maps

Repository map

Folder	Purpose
apps/	Source and binaries: buggy_app, leaky_app variants and perf_target variants
output/	initramfs_lab3.cpio.gz for QEMU
logs/	GDB, Valgrind, strace, perf, cyclictest, stress-ng and plot evidence
report/	Lab03_Report.pdf - full assignment report

Technical note: The remote GDB setup uses UART-to-TCP forwarding because the QEMU build in the reported experiment does not support SLIRP user networking.

02 / PART A

Remote Debugging with GDB

The host uses GDB to inspect a C program executing inside QEMU ARM through gdbserver. Breakpoints, variable inspection, registers and backtraces expose faulty control flow and memory use.

Three investigated programming errors

Issue	Observed cause	Debugging evidence
Array out-of-bounds	Loop uses `i <= len`	`i = 5, len = 5`
Use-after-free	Pointer accessed after `free()`	Returned pointer `0x8dda8`
Integer overflow	Signed 32-bit product exceeds valid range	Observed value: 1,410,065,408

Debugging approach

- Cross-compile the buggy C program for ARM and include it in the guest root filesystem.
- Run gdbserver on the ARM guest and connect from the Host using the forwarded serial/TCP path.
- Set breakpoints near suspicious code; inspect local variables, call stacks and register values.
- Relate observed runtime values to the source-level cause of each error.

Why integer overflow occurs

```
Mathematical result: 100000 x 100000 = 10,000,000,000
Signed 32-bit int maximum:            2,147,483,647
=> The result cannot be represented by signed 32-bit int.
```

Outcome: All three errors were localized and analyzed. The report does not claim that each error necessarily caused a crash. **Evidence:** logs/gdb_session.txt.

The exercise demonstrates why debugging a program that appears to run normally can still reveal invalid indexing, stale pointers and arithmetic behavior that violates type limits.

03 / PART B

Memory Analysis with Valgrind

The original leaky_app is tested on the x86_64 host, repaired by adding missing deallocation, and retested. ARM memory snapshots provide separate, limited runtime observations.

Before-and-after Valgrind measurements

Metric	Before fix	After fix
Definitely lost	16,900 bytes	0 bytes
Lost blocks	250	0
Allocations	352	352
Frees	102	352
Error contexts	3	0

Leak breakdown

Source of leaked allocations	Bytes
Name storage	2,100
Score storage	2,000
Message processing	12,800
Total	**16,900**

Valgrind after the fix: `ERROR SUMMARY: 0 errors from 0 contexts`. This validates the tested execution, not all possible program paths.

ARM memory checkpoints

Metric	Before processing	After processing
VmSize	776 KB	776 KB
VmRSS	4 KB	4 KB
VmData	152 KB	152 KB

The ARM probe uses two SIGSTOP checkpoints. Stable VmRSS does not prove leak-free execution; allocations may occur within an already mapped heap. The unusually low VmRSS values should be interpreted cautiously.

Evidence: logs/valgrind_report.txt, logs/valgrind_fixed.txt, logs/b3_qemu_console.log.

04 / PART C

System Calls & CPU Profiling

The perf_target workload combines sqrt()/log() computation, linear searching and file I/O (10,000 lines). strace measures syscall activity; perf samples CPU-intensive symbols and hardware counters.

strace summary - x86_64

System call	Calls	Share of syscall time
write	70	30.10%
execve	1	20.23%
munmap	3	12.85%
close / mmap	4 / 14	12.35% / 9.45%

122	1.206 ms	51.39%
Total system calls	Total syscall time	heavy_compute overhead

perf measurements - extended x86_64 workload

Metric	Recorded value
CPU samples / lost samples	395 / 0
Elapsed execution time	1.9806 seconds
CPU cycles / instructions	4,841,444,106 / 21,608,569,672
IPC	4.46
Cache misses / branch misses	488,237 / 13,534

ARM QEMU syscall experiment

Architecture	Execution time	Syscalls	Syscall time
ARMv7	33.68 seconds	100	17.364 ms

The x86 profiling binary performs 3,000 computational iterations (versus 50 in the original workload). Results must not be directly compared with the ARM QEMU 50-iteration run; emulator overhead further limits cross-platform interpretation.

Evidence: logs/strace_summary.txt, logs/perf_report.txt, logs/perf_stat.txt and logs/c3_qemu_console.log.

05 / PART D

Real-Time Scheduling Latency

cyclictest measures thread wake-up and scheduling delay on the AlmaLinux host, both without additional load and while stress-ng runs CPU, I/O and memory workers.

Measurement setup

Kernel	Threads	Policy / priority	Interval / loops
Linux 5.14 PREEMPT_DYNAMIC	4	SCHED_FIFO / 80	1,000 µs / 10,000 per thread

Latency per thread (microseconds)

Thread	Baseline min / avg / max	Under stress min / avg / max
0	7 / 14 / 121	6 / 12 / 2,042
1	6 / 14 / 124	4 / 10 / 1,121
2	8 / 15 / 142	4 / 10 / 964
3	7 / 13 / 188	6 / 19 / 3,919

188 µs
Baseline worst case

3,919 µs
Stress worst case

42 / 39,968
Stress samples >500 µs

Interpretation

- Baseline: none of the 40,000 recorded samples exceeded the 500 µs deadline.
- Under stress (16 CPU + 4 I/O + 2 memory workers), 42 of 39,968 samples exceeded 500 µs.
- Low mean latency does not guarantee deadlines; rare worst-case spikes matter in control systems.
- PREEMPT_RT may be worth evaluating for strict deadlines, but was not experimentally tested in this lab.

Chart: logs/latency_histogram.png. The stress histogram count is reported as recorded (39,968 samples). All timing results are from the host, not from QEMU.

06 / REPRODUCTION

Essential Run Commands

Run from the LAB-03 project root. The examples use the same binaries and tools described in the supplied README.

1. Memory leak verification (Host)

```
valgrind --leak-check=full --show-leak-kinds=all \
  ./apps/leaky_app
valgrind --leak-check=full --show-leak-kinds=all \
  ./apps/leaky_app_fixed
```

2. System-call and CPU profiling (Host)

```
strace -c ./apps/perf_target
perf record -g --call-graph fp -F 199 -e cpu-clock:u \
  -- ./apps/perf_target_profile
perf report --stdio
perf stat -e task-clock,cycles:u,instructions:u,\
cache-misses:u,branches:u,branch-misses:u \
  -- ./apps/perf_target_profile
```

3. QEMU ARM startup (Host)

```
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M \
  -smp 2 -display none -monitor none -serial stdio \
  -nic none \
  -kernel "$HOME/embedded_lab1/output/zImage" \
  -dtb "$HOME/embedded_lab1/output/vexpress-v2p-ca9.dtb" \
  -initrd "./output/initramfs_lab3.cpio.gz" \
  -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
```

4. Inside QEMU

```
/usr/bin/measure_leaky_arm.sh
/usr/bin/strace -c /usr/bin/perf_target_arm
```

5. Host scheduling-latency test

```
sudo /usr/bin/cyclictest --default-system --mlockall \
  --threads=4 --priority=80 --policy=fifo \
  --interval=1000 --loops=10000 --histogram=5000 \
  --histfile=/tmp/lab03_hist_recheck.txt --quiet
```

Note: The cyclictest experiment must be performed on the host. Repeated results can vary with machine load and scheduler state. The LAB-01 kernel and Device Tree must already exist for QEMU startup.

07 / EVIDENCE & CONCLUSIONS

Submission Checklist & Limits

Keep raw experimental outputs with the final report so the observations can be reproduced and audited.

Core artifacts

Area	Evidence files
GDB debugging	logs/gdb_session.txt
Memory analysis	logs/valgrind_report.txt; logs/valgrind_fixed.txt; logs/b3_qemu_console.log
CPU profiling	logs/strace_summary.txt; logs/perf_report.txt; logs/perf_stat.txt; logs/c3_qemu_console.log
Real-time latency	logs/hist_standard.txt; logs/hist_stress.txt; logs/d_latency_summary.txt; logs/latency_histogram.png
Final report	report/Lab03_Report.pdf

Important limitations

- Valgrind confirms allocation behavior in the tested host execution; ARM /proc snapshots do not independently prove the absence of leaks.
- perf is a sampling-based profiler; reported symbol overhead is not exact wall-clock execution share.
- ARM QEMU timings include emulation effects and should not represent physical Cortex-A9 performance.
- The tested host kernel is PREEMPT_DYNAMIC, not PREEMPT_RT; finite cyclictest runs cannot guarantee all future deadlines.
- The stress run captured 39,968 samples; this exact sample count is preserved in the reported results.

Conclusion

LAB-03 demonstrated a combined workflow for locating three C programming errors with remote GDB; reducing **16,900 bytes of confirmed leaks to zero** in the tested Valgrind run; identifying **heavy_compute()** as a dominant CPU hotspot; and measuring host scheduling behavior. Baseline scheduling stayed below 500 µs, while stress introduced latency spikes up to **3,919 µs**.

Main lesson: A correct-looking application still needs explicit memory diagnostics, syscall tracing, CPU profiling and worst-case latency evaluation. Conclusions should remain limited to the environment and workload actually measured.

Source basis: condensed English adaptation of the provided LAB-03 README (October 17, 2026). Reported experimental figures were preserved from that source.